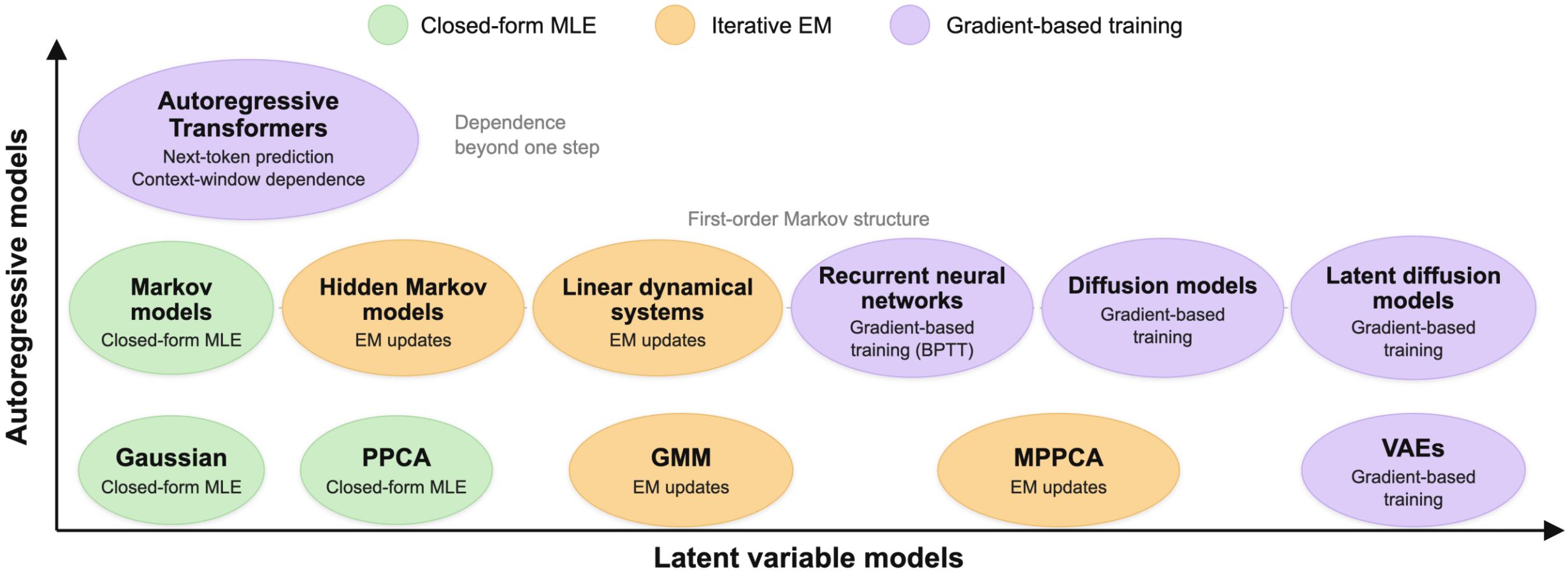
Deep Generative Models: State Space Models, Selective State Space Models & MAMBA

Fall Semester 2026

René Vidal
Director of the Center for Innovation in Data Engineering and Science (IDEAS),
Rachleff University Professor, University of Pennsylvania
Amazon Scholar & Chief Scientist at NORCE

Penn
University of Pennsylvania

Taxonomy of Generative Models



Penn
University of Pennsylvania

Selective State Space Models (Selective SSMs)

- State Space Models have overloaded meanings;
  - In particular, SSMs can mean linear dynamical systems

- LDSs (without noise)

$$\boldsymbol{z}_t = \boldsymbol{A}\boldsymbol{z}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t$$
$$\boldsymbol{y}_t = \boldsymbol{C}\boldsymbol{z}_t$$

- RNNs

$$\boldsymbol{z}_t = g(\boldsymbol{A}\boldsymbol{z}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t)$$
$$\boldsymbol{y}_t = f(\boldsymbol{C}\boldsymbol{z}_t)$$

- Selective SSMs

$$\boldsymbol{z}_t = \boldsymbol{A}_t\boldsymbol{z}_{t-1} + \boldsymbol{B}_t\boldsymbol{x}_t$$
$$\boldsymbol{y}_t = \boldsymbol{C}_t\boldsymbol{z}_t$$

  - $\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{C}_t$ depend on input $\boldsymbol{x}_t$
    - e.g., $\boldsymbol{A}_t = s(\boldsymbol{x}_t)$ for some learnable s
      - time-varying
  - Similar, if not identical, to gating

Why Selective SSMs?

- LDSs
  - simple, theoretically grounded
  - not expressive enough

- RNNs
  - inefficient (can’t be trained in parallel, can’t even be efficiently unroll for 1K+ tokens)
  - gradient exploding & gradient vanishing

- Selective SSMs
  - efficient parallel algorithms
  - increased expressivity by incorporating gating or selection mechanism
  - avoid gradient exploding by careful parametrization

Problem Setup

$$\boldsymbol{z}_t = \boldsymbol{A}_t \boldsymbol{z}_{t-1} + \boldsymbol{B}_t \boldsymbol{x}_t$$
$$\boldsymbol{y}_t = \boldsymbol{C}_t \boldsymbol{z}_t$$

$$\boldsymbol{A}_t = s_A(\boldsymbol{x}_t), \boldsymbol{B}_t = s_B(\boldsymbol{x}_t), \boldsymbol{C}_t = s_C(\boldsymbol{x}_t)$$

s_A, s_B, s_C are learnable

- An example (yet practical) choice:
  - $s_A(\boldsymbol{x}_t) = \text{diag}(\text{sigmoid}(\text{linear}(\boldsymbol{x}_t))$
  - $\text{sigmoid}(\xi) = \frac{1}{1+\exp(-\xi)} \in (0,1)$
    - prevents gradient explosion
    - crucial for stability
  - $\text{diag}(\cdot)$ makes $\boldsymbol{A}_t$ diagonal

- Assumption: $\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{C}_t$ are diagonal
  - lead to scalar dynamics
  - crucial for efficiency

- Goal: given initial state $\boldsymbol{z}_0$ and $\{\boldsymbol{x}_t, \boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{C}_t\}_{t=1,\dots,T}$, compute $\boldsymbol{y}_1, \dots, \boldsymbol{y}_T$ efficiently
  - this amounts to computing the forward pass of a SSM layer
  - during training, sequentially computing $\boldsymbol{y}_1, \dots, \boldsymbol{y}_T$ won't be scalable for large T

Scalar Dynamics

- We now consider a case where everything is a scalar

$$z_t = a_t z_{t-1} + b_t x_t$$
$$y_t = c_t z_t$$

- Goal: given initial state z_0 and $\{x_t, a_t, b_t, c_t\}_{t=1,\dots,T}$, compute $y_1, \dots, y_T$ efficiently
  - sequentially computing $y_1, \dots, y_T$ won't be scalable for large T

- Observation:
  - We only need to think about computing $z_1, \dots, z_T$ with as much parallelism as possible
    - given all c_t's and z_t's, we can compute y_t's in parallel
  - We can compute $d_t \coloneqq b_t x_t$ in parallel, so we

$$z_t = a_t z_{t-1} + d_t$$

Scalar Dynamics

$$z_t = a_t z_{t-1} + d_t$$

- **Simplified Case 1**: if $d_t = 0$, then we have
$$z_t = a_t a_{t-1} \cdots a_1 z_0$$
  - we just need to compute $[a_1, a_1 a_2 \ldots, a_1 a_2 \cdots a_T]$ (cumulative product)
- **Simplified Case 2**: if $a_t = 1$, then we have
$$z_t = z_0 + d_1 + \cdots + d_t$$
  - we just need to compute $[d_1, d_1 + d_2 \ldots, d_1 + d_2 + \cdots + d_T]$ (cumulative sum)

- In general, we can write the dynamics $z_t = a_t z_{t-1} + d_t$ as
$$\begin{bmatrix} z_t \\ 1 \end{bmatrix} = \begin{bmatrix} a_t & d_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} z_{t-1} \\ 1 \end{bmatrix}$$
  - We just need to compute the cumulative product of 2×2 matrices $\begin{bmatrix} a_t & d_t \\ 0 & 1 \end{bmatrix}$

Associative Operator

$$\begin{bmatrix} z_t \\ 1 \end{bmatrix} = \begin{bmatrix} a_t & d_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} z_{t-1} \\ 1 \end{bmatrix}$$

- Matrix product, scalar product, scalar addition are all associative operators $\bigoplus$

$$a \bigoplus b \bigoplus c = (a \bigoplus b) \bigoplus c = a \bigoplus (b \bigoplus c)$$

- Since

$$\begin{bmatrix} a_t & d_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a_\tau & d_\tau \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} a_t a_\tau & a_t d_\tau + d_t \\ 0 & 1 \end{bmatrix}$$

we can define the associative operator $\bigoplus$ this 2x2 matrix product as

$$(a_t, d_t) \bigoplus (a_\tau, d_\tau) = (a_t a_\tau, a_t d_\tau + d_t)$$

- Why we need this notation:
  - For any associative operator $\bigoplus$, parallel algorithms exist to compute

$$[\xi_1, \xi_1 \bigoplus \xi_2 \dots, \xi_1 \bigoplus \xi_2 \bigoplus \cdots \bigoplus \xi_T]$$

Hillis/Steele Associative Scan Algorithm

$$[\xi_1, \xi_1 \oplus \xi_2 \ldots, \xi_1 \oplus \xi_2 \oplus \cdots \oplus \xi_T]$$

step = 1, $p[i] = \xi_{i+1}$

while step < T: # assume T is some power of 2

    for i in range(step, T): # can run in parallel with p processors

        $p[i] = p[i - step] \oplus p[i]$

    step *= 2

sequential work [while loop]: $\log(T)$

total work: $\frac{T}{p} \cdot \log(T)$

Example (T=8): (the symbol $\oplus$ is omitted in interest of space)

Iter	Step	$p[0]$	$p[1]$	$p[2]$	$p[3]$	$p[4]$	$p[5]$	$p[6]$	$p[7]$
0	1	ξ_1	ξ_2	ξ_3	ξ_4	ξ_5	ξ_6	ξ_7	ξ_8
1	2	ξ_1	$\xi_1\xi_2$	$\xi_2\xi_3$	$\xi_3\xi_4$	$\xi_4\xi_5$	$\xi_5\xi_6$	$\xi_6\xi_7$	$\xi_7\xi_8$
2	4	ξ_1	$\xi_1\xi_2$	$\xi_1\xi_2\xi_3$	$\xi_1\xi_2\xi_3\xi_4$	$\xi_2\xi_3\xi_4\xi_5$	$\xi_3\xi_4\xi_5\xi_6$	$\xi_4\xi_5\xi_6\xi_7$	$\xi_5\xi_6\xi_7\xi_8$
3	8	ξ_1	$\xi_1\xi_2$	$\xi_1\xi_2\xi_3$	$\xi_1\xi_2\xi_3\xi_4$	$\xi_1\xi_2\xi_3\xi_4\xi_5$	$\xi_1\xi_2\xi_3$ $\xi_4\xi_5\xi_6$	$\xi_1\xi_2\xi_3$ $\xi_4\xi_5\xi_6\xi_7$	$\xi_1\xi_2\xi_3\xi_4$ $\xi_5\xi_6\xi_7\xi_8$

Forward Pass of Selective SSMs (Vector Case)

$$\boldsymbol{z}_t = \boldsymbol{A}_t \boldsymbol{z}_{t-1} + \boldsymbol{B}_t \boldsymbol{x}_t$$
$$\boldsymbol{y}_t = \boldsymbol{C}_t \boldsymbol{z}_t$$

$$\boldsymbol{A}_t = s_A(\boldsymbol{x}_t), \boldsymbol{B}_t = s_B(\boldsymbol{x}_t), \boldsymbol{C}_t = s_C(\boldsymbol{x}_t)$$

s_A, s_B, s_C are learnable

- An example (yet practical) choice:
  - $s_A(\boldsymbol{x}_t) = \mathrm{diag}(\mathrm{sigmoid}(\mathrm{linear}(\boldsymbol{x}_t))$
  - $\mathrm{sigmoid}(\xi) = \frac{1}{1+\exp(-\xi)} \in (0,1)$
    - prevents gradient explosion
    - crucial for stability
  - $\mathrm{diag}(\cdot)$ makes $\boldsymbol{A}_t$ diagonal

- Given initial state $\boldsymbol{z}_0$ and $\{\boldsymbol{x}_t\}_{t=1,\dots,T}$
  1. compute the diagonal matrices $\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{C}_t$ (for all t in parallel):
     - $\boldsymbol{A}_t = s_A(\boldsymbol{x}_t), \boldsymbol{B}_t = s_B(\boldsymbol{x}_t), \boldsymbol{C}_t = s_C(\boldsymbol{x}_t)$
  2. compute all vector outputs $\boldsymbol{y}_1, \dots, \boldsymbol{y}_T$ via associative scan algorithms